

1. Introducción a SQL: SQL Es un lenguaje diseñado para trabajar con bases de datos relacionales.

- **Estructura Básica:**

- **Tabla:** Guarda información (como una hoja de Excel pero con reglas), por ejemplo, "Clientes" o "Reservas".
- **Fila / Registro:** Representa un elemento concreto (un cliente específico).
- **Columna / Campo:** Es el tipo de dato que se guarda (nombre, precio, fecha).
- **Relación:** Conecta una tabla con otra (ej. una reserva se asocia a un cliente y a una habitación).

- **Comentarios:** -- comentario (con espacio), #comentario o /* comentario largo */.

2. DDL (Definición de Estructura-Arquitectura)

El DDL se encarga de definir y modificar la estructura de la base de datos y sus objetos principales (tablas, vistas, etc.). Se utilizan los **comandos CREATE, ALTER y DROP** para montar el esqueleto donde luego se guardará la información.

Pasos Iniciales, Operaciones, Tipos de Datos y Restricciones

- **Crear y Usar:** CREATE DATABASE nombre; y USE nombre;
- **Creación de Tablas y Restricciones** -> CREATE TABLE nombre_tabla (...).
- **Tipos comunes:**
 - INT / BIGINT: Números enteros.
 - DECIMAL(10,2): Ideal para dinero o precios (mejor que FLOAT).
 - VARCHAR(50) / TEXT: Texto corto / Texto largo.
 - DATE / DATETIME / TIMESTAMP: Para almacenar fechas y horas.

- **BOOLEAN:** Verdadero/falso (en MariaDB suele ser un alias de TINYINT(1)).
 - **ENUM ('A', 'B'):** Valores cerrados específicos.
 - **Restricciones clave / claves de arquitectura:**
 - **PRIMARY KEY:** Identifica de forma única cada fila.
 - **NOT NULL:** Hace que el campo sea obligatorio.
 - **UNIQUE:** Evita que el dato se repita en otra fila (ej. correos).
 - **DEFAULT:** Asigna un valor por defecto si no se especifica otro.
 - **AUTO_INCREMENT:** El identificador numérico se genera solo.
 - **Claves Foráneas (FOREIGN KEY):** Obligan a que un dato (ej. id_cliente) exista previamente en su tabla original, evitando crear registros "fantasma". Se señalan mediante:
 - **CONSTRAINT** fk_nombre_relacion
 - **FOREIGN KEY** (nombre_nuevaTabla_campo)
 - **REFERENCES** nombre_tablaDePK (nombre_campo)
 - **ON UPDATE/DELETE CASCADE:** Se añade seguido a la declaración de una FK.
 - **Modificar (ALTER **TABLE**):** Permite añadir (**ADD**), modificar el tipo (**MODIFY**), renombrar (**CHANGE**) o borrar (**DROP COLUMN**) **columnas** existentes.
 - **Borrar Estructuras (DROP):** DROP TABLE elimina una tabla entera y DROP DATABASE elimina la base de datos completa.
-

3. DML (Manipulación de Datos)

El DML se encarga de gestionar el ciclo de vida de un dato, y permite alterar la información interna de las tablas. Son las **operaciones** de escritura del acrónimo **CRUD** (**Create, Update, Delete**), las cuales se apoyan en la cláusula **WHERE**.

Operaciones Básicas (CRUD)

- **Insertar (INSERT INTO):** Añade nuevos registros (filas) permitiendo hacer un "Insert normal" para una fila o un "Insert múltiple" para varias filas a la vez.

Regla práctica: Insertar primero en las tablas "padre" (Clientes) y luego en las "hijas" (Reservas).

(Para hacer un insert, puede o no indicarse el listado de columnas, pero de una u otra forma, el orden de values siempre ha de coincidir con el de las columnas).

- **Actualizar (UPDATE):** Modifica filas concretas **mediante el uso de SET**.
Peligro: Si no se pones la condición **WHERE**, se actualizará absolutamente TODOS los registros de la tabla.
- **Borrar (DELETE):** Elimina filas concretas. **Peligro:** Igual que el anterior, sin un **WHERE**, borrará toda la información de la tabla.

(ES BUENA PRÁCTICA PROBAR LAS OPERACIONES DE UPDATE Y DELETE PREVIAMENTE CON UN SELECT).

4. DQL (Data Query Language) – Consulta de datos

Abarca exclusivamente la extracción y lectura de datos sin modificarlos.

El comando **SELECT** es el núcleo del lenguaje DQL. Dominarlo significa poder transformar una masa de datos inertes en información útil, comprensible y accionable que permite a los analistas formular preguntas con precisión. *(En entornos empresariales casi todas las decisiones basadas en datos comienzan con una consulta **SELECT**).*

La potencia del select radica en su flexibilidad, ya que permite combinaciones con filtros, ordenaciones o cálculos.

La estructura básica de una consulta SELECT es:

SELECT	+	FROM	+	WHERE
(qué información		(de dónde		(condición
- columna)		- tabla)		- filas)

- **Cláusulas de una consulta SELECT:** En SQL, una cláusula se define como un bloque lógico de instrucciones que forma parte de una consulta o sentencia mayor:

- **SELECT:** (Obligatoria) Define qué columnas, cálculos o funciones matemáticas quieres extraer y mostrar.
- **FROM:** (Obligatoria) Indica la tabla o tablas principales de donde provienen los datos originales.
- **JOIN:** (Suele ir acompañada de ON) Se utiliza para vincular y cruzar datos con otras tablas basándose en una relación (como las claves foráneas).
- **WHERE:** Filtra los registros individuales basándose en condiciones lógicas antes de que se realice cualquier agrupación.
- **GROUP BY:** Agrupa las filas que tienen los mismos valores en columnas específicas y aplicar func. agregadas a los grupos. (Muy usado con funciones como COUNT o SUM).

(Regla: todo campo que aparece en el SELECT y no es una func. agregada, debe de aparecer dentro de GROUP BY).

- **HAVING:** Funciona como un WHERE, pero se aplica **después de haber agrupado los datos** (ej. mostrar solo los clientes con 2 o más pedidos).
- **ORDER BY:** Ordena el bloque final de resultados de forma ascendente (ASC) o descendente (DESC) según una o varias columnas. (Se usa al final de la consulta).
- **LIMIT** (y su acompañante OFFSET): Recorta el resultado final para mostrar solo una cantidad específica de filas (y opcionalmente, desde qué fila empezar a contar).

- **Funciones Agregadas -> Resúmenes**

Las funciones agregadas se pueden utilizar:

- **En el SELECT** (Para mostrar el resultado).
- **En el HAVING** (Para filtrar los grupos).
- **En el ORDER BY** (Para ordenar por el resumen).

PERO NUNCA EN EL WHERE.

Las 5 más usadas son:

- **COUNT** (contar filas o valores)
- **SUM** (sumar)
- **AVG** (promedio)
- **MIN** (mínimo)
- **MAX** (máximo).

- La cláusula **WHERE FILTRA ANTES DE AGRUPAR**, y en ella entran en juego los operadores:

- **De comparación:** = != < > <= >=
- **Lógicos:** AND, OR, NOT e IS NULL.
- **Avanzados:**
 - **IN:** Para buscar coincidencias dentro de una lista
 - **BETWEEN:** Para buscar valores dentro de un rango.
 - **DISTINCT:** Para evitar duplicados.
 - **LIKE:** Para búsquedas por patrones de texto usando % (cualquier cantidad de caracteres) y _ (un solo carácter).
 - comodines %%%
 - que empiece por m -> "m%"
 - que acabe por m -> "%m"
 - que contenga la m -> "%m%"

- **JOINS (Consultas relacionales):** Permiten unir la información de varias tablas en una sola consulta. Mediante una relación lógica basada en una clave primaria y una clave foránea.

(Las joins reconstruyen la información que fue separada por la normalización).

- **INNER JOIN:** Sólo devuelve las coincidencias.
- **LEFT JOIN:** Devuelve la información de la tabla principal aunque no haya coincidencias en la secundaria. *Detecta registros sin actividad.*
- **RIGHT JOIN:** Lo contrario que left. *Detecta registros huérfanos*
- **FULL JOIN:** Devuelve todas las filas de ambas tablas, uniendo cuando hay coincidencias y mostrando NULL cuando no las hay.

Aunque PostgreSQL y SQL Server sí soportan FULL JOIN, MySQL no, y para hacerlo en este motor habría que hacer:

LEFT JOIN (todo A + coincidencias de B)

RIGHT JOIN (todo B + coincidencias de A)

UNION para no duplicar lo que coincide en ambos.

Ej. **SELECT** columnas
 FROM A
 LEFT JOIN B **ON** A.clave = B.clave

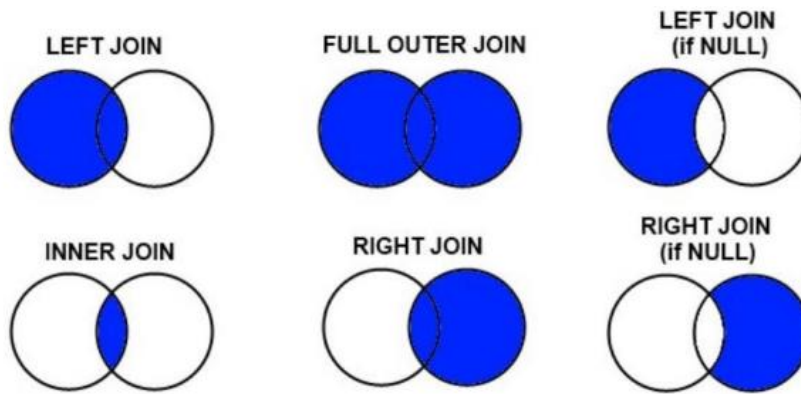
UNION

SELECT columnas
 FROM A
 RIGHT JOIN B **ON** A.clave = B.clave;

(UNION es lento porque necesita ordenar y comparar. La solución eficiente en este caso sería usar UNION ALL y añadir un filtro en la segunda join “WHERE A.clave IS NULL”).

(Importante: las columnas deben ser las mismas (mismo número, orden y tipos)).

Además de estos tipos de JOINS, existen otros como son:



- **Subconsultas (IN y EXISTS):** Una subconsulta es una consulta anidada dentro de otra: Sirve para filtrar, calcular o incluso generar tablas intermedias sobre las que poder operar.

La sintaxis más común es encontrar la subconsulta dentro de una cláusula WHERE, pero también puede usarse en FROM (como tabla virtual) o incluso en SELECT (para calcular valores derivados).

Las subconsultas sirven para:

- Filtrar resultados con valores de otra consulta.
- Comprobaciones con IN y EXISTS.
- Consultas anidadas.
- Combinar con UNION/UNION ALL.

Regla:

- IN: cómodo para listas.
- EXISTS: suele ir muy bien cuando hay muchas filas y buenas claves/index.

Ej. -> pasajeros que tengan reservas de precio > 500

```
SELECT nombre
FROM Pasajeros
WHERE id_pasajero IN (
  SELECT id_pasajero
  FROM Reservas
  WHERE precio > 500
);
```

*1º saco los id_pasajero que cumplen la condición,
luego filtro la tabla grande con esos ids.*

```
SELECT P.nombre
FROM Pasajeros P
WHERE EXISTS (
    SELECT 1
    FROM Reservas R WHERE R.id_pasajero
    = P.id_pasajero
    AND R.precio > 500
);
```

EXISTS no necesita la lista, solo pregunta: ¿existe al menos una fila?

5. TCL (Transaction Control Language) - Control de Transacciones

Una transacción es un conjunto de operaciones DML que se tratan como una única unidad de trabajo. (O se ejecutan todas las operaciones con éxito, o no se ejecuta ninguna).

Las transacciones son la base de la fiabilidad de cualquier sistema que trabaje con dinero, stocks, reservas, notas, etc.

- **El Modelo ACID garantiza la fiabilidad de las transacciones.**

Toda transacción se apoya en 4 propiedades clave para garantizar su fiabilidad:

- **A (Atomicidad):** "Todo o nada".
- **C (Consistencia):** La base de datos siempre pasa de un estado válido a otro estado válido, respetando todas las reglas y restricciones.
- **I (Isolation/Aislamiento):** Si hay varias transacciones a la vez, cada una se comporta como si estuviera sola.
- **D (Durabilidad):** Tras confirmar (COMMIT), la transacción persiste aunque falle el hardware.

- **Comandos de Control en MySQL**

- **START TRANSACTION (o BEGIN):** Marca el inicio. El motor toma nota, pero no hace cambios permanentes.
- **COMMIT:** Confirma los cambios, haciéndolos permanentes.
- **ROLLBACK:** Deshace los cambios, devolviendo la base de datos a su estado original.
- **SAVEPOINT nombre:** Crea un punto de control intermedio. Permite hacer un ROLLBACK TO SAVEPOINT nombre para deshacer un error.

- **Autocommit**

Por defecto, MySQL tiene el autocommit activado. Esto significa que cada comando INSERT o UPDATE individual se confirma automáticamente sin posibilidad de hacer ROLLBACK, por lo que es necesario usar siempre START TRANSACTION de forma explícita para poder encadenar varias operaciones.

- **Gestión de errores:** MySQL **NO** hace un ROLLBACK automático si falla una sentencia. Lo que hace es devolver un error y detener la sentencia, pero deja las correctas siguen en memoria y la transacción abierta, y es el programador el que debe decidir si ejecuta un ROLLBACK manual o soluciona el error.

Handler: En la práctica, para que ese error no deje la transacción colgada, ejecutamos nuestras consultas dentro de un Procedimiento Almacenado donde configuramos un **DECLARE EXIT HANDLER**. Este Handler es un bloque de código que está "escuchando". Su instrucción es simple: *"Si escuchas que ocurre cualquier error (como el grito del Trigger), detén todo y ejecuta un ROLLBACK inmediatamente"*.

- **Problemas de Concurrencia (Si falla el aislamiento):**

- **Lectura Sucia (Dirty Read):** Leer datos no confirmados que podrían sufrir un ROLLBACK.
- **Lectura No Repetible:** Un mismo SELECT da datos distintos en la misma transacción porque otro usuario hizo un UPDATE.

- **Lectura Fantasma:** Aparecen filas nuevas al repetir un SELECT porque otro usuario hizo un INSERT.

Esos problemas se pueden controlar mediante bloqueos (locks) y niveles de aislamiento (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ y SERIALIZABLE).

A mayor aislamiento, más seguridad frente a anomalías pero mayor impacto sobre el rendimiento).

En un entorno de desarrollo real, cuando se diseña el backend de una aplicación siguiendo el patrón Model View Controller, el control de las transacciones rara vez se hace con puro SQL en la base de datos.

Lo estándar es gestionar la transacción directamente desde el código de la aplicación (por ejemplo, usando métodos de la API de JDBC en Java dentro de las clases DAO de acceso a datos). El flujo de trabajo normal es este:

1. El código Java desactiva el autocommit (esto equivale a un START TRANSACTION).
2. El código envía el INSERT o UPDATE a MySQL.
3. Ese INSERT hace que el Trigger de MySQL reaccione de forma invisible en segundo plano.
4. Si el Trigger detecta una infracción, le devuelve una "Excepción" (un error) a tu programa.
5. El bloque catch en Java captura ese error y es tu código el que ejecuta la orden de ROLLBACK. Si todo va bien, ejecuta el COMMIT.

6. DCL (Data Control Language) - Controlar accesos

El DCL gestiona la seguridad, los permisos, los roles y los usuarios del sistema.

Usuario: puede conectarse a una BD y representa a una persona o una aplicación. Cada usuario tiene sus propias credenciales (usuario / contraseña) y actúa con un determinado conjunto de permisos.

Privilegio: permiso para realizar una acción

Rol: conjunto de privilegios bajo una denominación que puede ser asignado a varios usuarios.

Modelo de mínimo privilegio: afirma que cada usuario debe tener sólo los privilegios estrictamente necesarios para hacer su trabajo, reduciendo el riesgo a errores, abusos internos y el impacto de posibles ataques.

- **Comandos principales:**

- **CREATE USER:** Crear usuarios en el sistema. (Ej. *CREATE USER 'medico1'@'%' IDENTIFIED BY 'Medico123!';*)
- **GRANT:** Otorgar permisos o roles. (Ej. *GRANT SELECT ON clinica_seguridad.pacientes TO 'rol_medico';*)
- **REVOKE:** Retirar permisos.
- **SHOW GRANTS:** Ver permisos actuales.

RBAC (Control de Acceso Basado en Roles): Simplifica la seguridad. En lugar de dar permisos persona a persona, se dan a un rol y se asigna la persona al rol.

- **Creación de Roles:** *CREATE ROLE 'rol_medico';* Crea el grupo vacío.
- **Vincular:** *GRANT 'rol_medico' TO 'medico1'@'%';* El usuario hereda las reglas del rol.

Orden de trabajo con roles y usuarios:

- El DBA crea usuarios.
- “ crea o reutiliza roles.
- “ asigna privilegios a roles.
- “ asigna roles a usuarios.
- “ audita y revisa periódicamente que los permisos sigan teniendo sentido.

La gran ventaja: Si mañana contratas a 50 médicos más, solo tienes que crear sus usuarios y asignarles el rol. No tienes que volver a configurar los permisos de las tablas uno por uno.

Implementar borrado lógico:

Para implementar el borrado lógico con una columna `deleted_at`, el proceso se divide en tres pasos prácticos que tocan los diferentes lenguajes (DDL, DML y DQL).

- **1. Modificar la estructura (DDL)**

Se añade la columna `deleted_at` en la tabla. Lo ideal es que sea de tipo fecha/hora para saber cuándo se eliminó, y que por defecto esté vacía (NULL), lo que significa que el registro está "activo".

```
ALTER TABLE clientes ADD COLUMN deleted_at DATETIME DEFAULT NULL;
```

- **2. Ejecutar el borrado lógico (DML)**

Cuando al método de la clase DAO reciba la orden de eliminar cliente no ejecutará un DELETE. En su lugar, lanzará un UPDATE guardando la fecha y hora exacta de ese momento.

-- En MySQL, NOW() captura la fecha y hora actual del sistema

```
UPDATE clientes SET deleted_at = NOW() WHERE id_cliente = 5;
```

- **3. Adaptar las consultas (DQL)**

Este es el paso crítico para que la "ilusión" funcione: a partir de ahora, todas las consultas de lectura deben filtrar la información para traer **solo** los registros cuya columna de borrado siga vacía.

```
SELECT * FROM clientes WHERE deleted_at IS NULL;
```

Optimización de consultas

Es reducir el coste de ejecución de una consulta manteniendo exactamente el mismo resultado. El "coste"? incluye el tiempo de ejecución, la memoria usada, la carga de la CPU y el número total de operaciones.

Los motores de BBDD crean automáticamente índices sobre las PK y las FK, pero **es responsabilidad del profesional forzar al motor de la BD a elegir la ruta**

óptima definiendo índices adicionales **en las columnas que más se utilizan para filtrar o para unir**.

Sintaxis: CREATE INDEX nombre_indice ON tabla (columna);

El coste oculto: Aunque aceleran las consultas, ocupan más espacio en disco y hacen que **la escritura sea más lenta**. Si insertas datos (INSERT), la base de datos también tiene que actualizar el índice. Por tanto, tener *demasiados* índices ralentizará enormemente las operaciones de escritura.

El Enemigo: Full Table Scan

- **¿Qué es?:** Ocurre cuando la base de datos tiene que recorrer y leer **TODAS** las filas de una tabla una por una.
- **El Problema:** Su coste depende del tamaño de la tabla; es rápido con 1.000 filas, pero un completo desastre de rendimiento con 10 millones.
- **¿Cómo detectarlo?:** Utilizando el comando EXPLAIN antes de la consulta.

El comando **EXPLAIN** (o EXPLAIN ANALYZE según el motor de la BD) revela cómo el motor de la BD planea ejecutar una consulta, indicando qué tablas va a recorrer, q tipo de unión usará, y si aprovechará un índice o realizará un escaneo completo permitiendo **identificar los cuellos de botella**.

```
• -- EXPLAIN muestra cómo piensa ejecutar MariaDB la consulta.
• -- Las columnas más importantes para esta práctica son:
• -- - type: forma de acceso a la tabla
• -- - possible_keys: índices posibles
• -- - key: índice realmente usado
• -- - rows: filas estimadas a examinar
• -- - Extra: información adicional
•
• -- type = ALL,   recorre toda la tabla
• -- type = range, usa un rango del índice
• -- type = ref,   usa un índice no único
• -- type = const, acceso muy eficiente a una fila concreta
```

```
• -- Antes de crear índices adicionales, revisamos cuáles existen.
• -- En este punto normalmente solo veremos los de PRIMARY KEY.
• SHOW INDEX FROM socios;
• SHOW INDEX FROM prestamos;
•
```

```

• -- CREAR ÍNDICES PARA OPTIMIZAR
•
• -- Índice para búsquedas exactas por email.
• -- Es muy útil para consultas tipo:
• -- WHERE email = '...'
• CREATE INDEX idx_socios_email ON socios(email);
•
• -- Índice para filtros por ciudad.
• -- Útil cuando se consulta mucho por ciudad.
• CREATE INDEX idx_socios_ciudad ON socios(ciudad);
•
• -- Índice para la clave foránea en prestamos.
• -- Ayuda en JOIN y en búsquedas de préstamos por socio.
• CREATE INDEX idx_prestamos_id_socio ON prestamos(id_socio);
•
• -- Índice para búsquedas por estado.
• -- Útil en consultas como WHERE estado = 'PENDIENTE'
• CREATE INDEX idx_prestamos_estado ON prestamos(estado);
•
• -- Índice para filtros por fecha.
• -- Ayuda especialmente en consultas con BETWEEN o >= <=
• CREATE INDEX idx_prestamos_fecha_prestamo ON
prestamos(fecha_prestamo);
•
• -- Índice compuesto.
• -- Este índice está pensado para consultas donde primero se filtra
• -- por id_socio y después por fecha_prestamo.
• -- El orden de las columnas dentro del índice importa.
• CREATE INDEX idx_prestamos_socio_fecha ON prestamos(id_socio,
fecha_prestamo);
•
• -- REPETIR EXPLAIN DESPUÉS DE OPTIMIZAR
•
• -- Ahora repetimos los EXPLAIN.
• -- La idea es comparar:
• -- - si aparece un índice en possible_keys
• -- - si el optimizador realmente lo usa en key
• -- - si baja el número de filas estimadas en rows
• -- - si cambia type de ALL a ref/range/const
•

```

Reglas de Oro para Optimizar Consultas

1. **Evitar SELECT *:** Nunca traigas datos que no vas a usar. Selecciona solo las columnas necesarias (ej. SELECT nombre, email) para gastar menos memoria y ser más rápido.

2. **Usar índices correctamente:** Créalos solo en las columnas que se filtran o relacionan mucho.
3. **Optimizar el WHERE:**
 - *Mal:* Usar funciones en la columna, como WHERE YEAR(fecha) = 2025 (impide usar el índice).
 - *Bien:* Usar rangos directos, como WHERE fecha BETWEEN '2025-01-01' AND '2025-12-31'.
4. **Optimizar los JOIN:** Todo campo que utilices en una condición JOIN debería estar siempre indexado para evitar comparar fila por fila.
5. **Limitar resultados:** Usar LIMIT (ej. LIMIT 10) reduce drásticamente la carga si solo necesitas ver unos pocos registros.

OJO -> Para usar INDEX hay que asegurarse de que las columnas de unión estén indexadas en ambas tablas.

Otra técnica para optimizar consultas es limitar los resultados cuando sea posible.

SEGURIDAD EN BBDD (auditoría, backups, restauración y protecc. de datos)

El enfoque de “**SEGURIDAD EN PROFUNDIDAD**” se basa en anticipar riesgos, monitorear acciones, crear defensas múltiples y planificar cómo recuperarte si algo falla.

- **1ª capa: Auditoría**

No es simplemente guardar archivos de registro (logs), sino mantener un sistema que permita reconstruir lo ocurrido, detectar comportamientos anómalos y demostrar el cumplimiento legal de normativas.

- **¿Qué registra?:** Quién accede, qué acciones realiza y en qué momento exacto lo hace.

- **Eventos clave a registrar:** Logins correctos e incorrectos, consultas a tablas sensibles, cambios en los permisos y operaciones críticas (como UPDATE o DELETE).
- **Sin auditoría:** Se pierde la trazabilidad, es imposible investigar incidentes y no se pueden cumplir las normativas legales.

(Los SGBD modernos permiten activar auditorías automáticas).

- **2ª capa: Resiliencia**

Depende de una buena estrategia de backups y restauración.

Los **tipos de backup** más comunes son:

- **Completo.**
- **Diferencial** -> Copia los datos desde el último backup completo.
- **Incremental** -> Copia sólo los cambios desde el último backup de cualquier tipo.

La restauración se mide con dos **métricas**:

- **RPO** (Recovery Point Objective) -> Cuántos datos puedes perder medidos en tiempo.
- **RTO** (rec Time ob) -> Cuánto tiempo se tarda en recuperar el sistema.

- **3ª capa: Protección de datos**

Protege incluso en caso de fuga o ataque.

Aquí entran en juego:

- **Encriptación en reposo** -> Archivos cifrados con TDE.
- **Encriptación en tránsito** -> Conexión cifrada con TLS.
- **Anonimización** -> En Estudios estadísticos, Big Data, Machine Learning, Informes; se Elimina permanentemente cualquier dato que permita identificar a la persona, dejando solo lo útil para el análisis.
- **Pseudoanonimización** -> En sistemas internos confidenciales, Hospitales, Bancos; se sustituyen los datos identificativos (ej. un DNI) por un ID inventado, guardando la relación real en una tabla secreta separada.

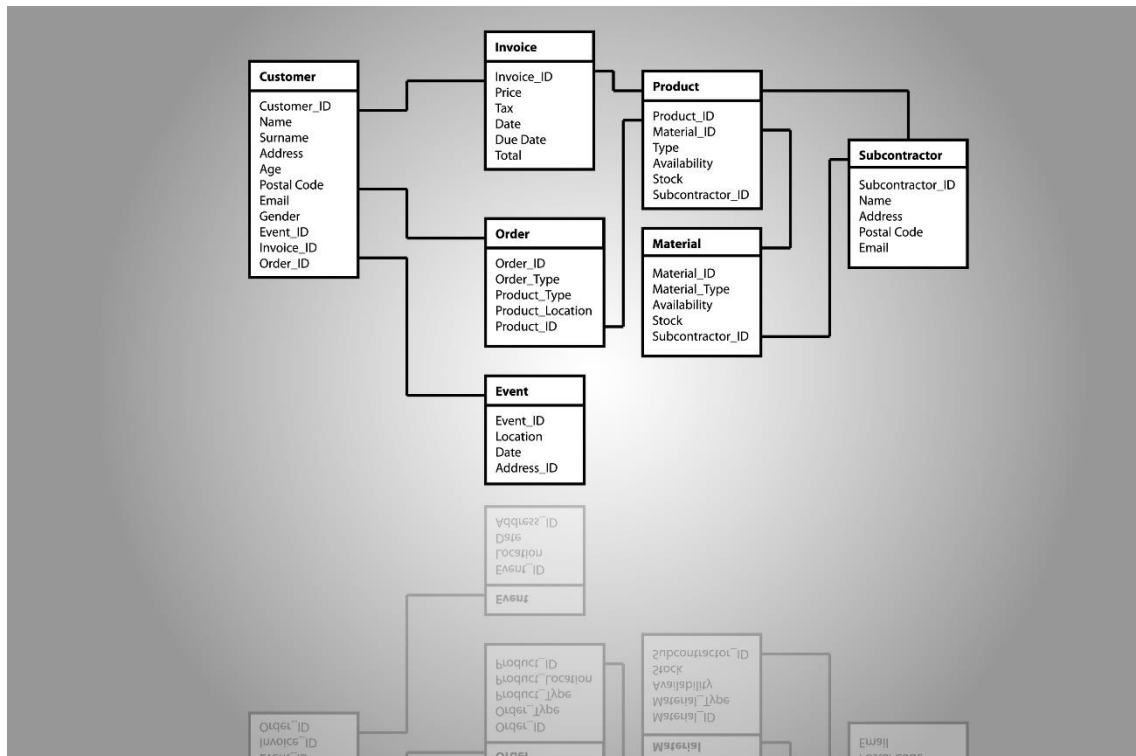
- **Data masking** -> En aplicaciones, Interfaces de usuario (UI), Sistemas de atención al cliente; se oculta la información de forma únicamente visual (ej. Mostrar solo los últimos 4 dígitos de una tarjeta: **** 4567).

**Herramientas nativas para backups:*

- MySQL / MariaDB -> *mysqldump, mysqlpump*
 - PostgreSQL -> *pg_dump, pg_basebackup*
 - SQL Server -> *SSMS o T-SQL*
 - Oracle -> *RMAN.*
-

¿Por qué en relaciones N:M se usa una Clave Primaria (PK) Compuesta?

- **El concepto:** La tabla intermedia en una relación Muchos a Muchos (N:M) no representa un objeto nuevo, sino la relación entre dos objetos existentes (ej. Un *Alumno* y un *Curso*).
- **La Identidad:** La combinación de las dos Claves Foráneas (FK) es la identidad real de ese registro y es única en el mundo real.



- **Razones para usar PK Compuesta:**
 1. La combinación de las dos FKs ya identifica de forma única la fila.
 2. Evita la creación de registros duplicados automáticamente.
 3. Es más eficiente (1 índice compuesto en vez de 2).
 4. Es más lógico porque no introduce datos que no aportan valor.

¿Qué pasa si usas una PK Artificial en una relación N:M pura?

Si creas un "ID inventado" (como id_matricula) para una tabla que solo relaciona dos objetos:

- **El Problema:** La base de datos no sabrá que es la misma matrícula repetida, permitiendo insertar duplicados lógicos (ej. el mismo alumno en el mismo curso dos veces) porque la clave primaria es el ID artificial y no la relación.

- **La "Solución" ineficiente:** Tendrías que crear una restricción UNIQUE adicional en los campos (id_alumno, id_curso) para evitar el error.
- **Consecuencia:** Terminas con más columnas, más índices (PK y UNIQUE), más complejidad y menor rendimiento para lograr el mismo resultado que con una PK Compuesta.

¿Cuándo SÍ se usa una PK Artificial?

Solo se debe utilizar cuando la tabla intermedia tiene vida propia o datos propios importantes.

- **El caso:** Cuando cada registro (fila) es una entidad en sí misma y tiene sentido referenciarla individualmente.
- **Ejemplos:**
 - **Detalle de un pedido:** Un producto dentro de un pedido tiene información clave como cantidad, precio_unitario o descuento. Además, el mismo producto podría repetirse en el mismo pedido con distintas condiciones promocionales.
 - **Asignaciones con histórico:** Un Profesor asignado a un Departamento con fecha_inicio y fecha_fin. El profesor podría volver al mismo departamento en el futuro, por lo que una PK Compuesta bloquearía esa segunda inserción.

Resumen de Casos Prácticos

Caso	Tipo de PK Recomendada	Razón Principal
Alumno - Curso (Matrícula)	PK Compuesta (id_alumno, id_curso)	Solo relaciona. Evita el duplicado lógico de que un alumno se matricule dos veces en el mismo curso.
Actor - Película (Reparto)	PK Compuesta (id_actor, id_pelicula)	Relación estándar. El actor participa una sola vez en la película.
Usuario - Rol (Asignación)	PK Compuesta (id_usuario, id_rol)	Relación pura. No tiene sentido que el mismo usuario tenga el mismo rol dos veces.
Pedido - Producto (Detalle)	PK Artificial (ej. id_detalle)	Es una "línea de venta" con datos propios importantes (precio, cantidad, descuentos).
Profesor - Depto. (Histórico)	PK Artificial (ej. id_asignacion)	Al guardar historial de fechas, el mismo profesor puede estar en el mismo departamento en distintos intervalos de tiempo.